

TERRAFORM STUDY SERIES · VOLUME TWO

Level 2 — Advanced IaC

Variables · Outputs · Workspaces · Multi-VPC

From one config, many environments. Every variable-passing mechanism, the networking model behind every `aws_security_group`, workspaces vs folders, count vs for_each, the secrets/outputs contract, and a full end-to-end walkthrough.

A TECHNICAL BOOK · INK & AMBER SERIES
COURSE MATERIAL BY **ENG. MOHAMED MAGDY**
VOLUME I = LEVEL 1 · VOLUME II = LEVEL 2

DAY 1

DAY 2 · 3

CAPSTONE

Map of the Book

Three parts, eight chapters, one capstone. Click any chapter to jump; every page has a **TOC** badge to come back.

I FRONT MATTER

01	Cover & Title	read first
02	Contents (this page)	you are here
03	Executive Summary	5 min

II PART I — DAY 1: THE DEEP STACK

04	Variables — precedence, types, validation	8 min
05	Functions, dynamic blocks, secrets	8 min
06	Networking — protocols, CIDR, routing	8 min
07	SG vs NACL, DNS, endpoints, peering	8 min
08	Workspaces — CLI, patterns, proof	7 min
09	Workspaces — CI, sharing, gotchas	7 min

III PART II — DAY 2: SCALING UP

10	Multi-Env — workspaces vs folders	8 min
11	Multi-VPC — the 3-VPC layout	8 min
12	count vs for_each — the rename trap	8 min
13	Outputs — the production contract	8 min

IV PART III — DAY 3 & CAPSTONE

15	ALB Walkthrough — two tiers, one codebase	10 min
14	End-to-End Project Walkthrough	10 min
16	★ Lab 1 — Multi-Env & Multi-VPC Build	15 min

V FINALE

17	Cheat · Quiz · Glossary · Resources	end
----	-------------------------------------	-----

The Route — One Page, Whole Book

Level 2 takes working single-env code and makes it **multi-env, multi-VPC, production-shaped**. Prereq: Level 1; everything runs on the local emulator.

PROBLEM 1

One code, many envs

Same HCL drives dev / staging / prod — per-env state, sizing, CIDRs that never overlap.

PROBLEM 2

Variables done right

7-mechanism precedence ladder, type system, validation, sensitive vs ephemeral, 20 key functions.

PROBLEM 3

Networking that works

TCP/UDP, CIDR math, route tables, stateful vs stateless, peering, endpoints.

PROBLEM 4

Outputs as contract

Module → stack, stack → stack, sensitive vs ephemeral vs Vault, CI playbook.

One sentence per chapter

Variables

Every way to pass a value in — the 7-step ladder, types, validation.

Functions & secrets

The 20 functions you'll call, dynamic blocks, Vault/SSM over tfvars secrets.

Networking I

TCP/UDP, handshake failures, CIDR math, reserved IPs, routing, tiers.

Networking II

SG vs NACL, DNS flags, endpoints, peering + the silent DNS bug.

Workspaces

One folder, one backend, many state files — CLI, patterns, isolation proof.

Workspaces in prod

Decision table, CI with TF_WORKSPACE, sharing, gotchas.

Multi-Env

Workspaces vs folders vs buckets — pick by blast radius.

Multi-VPC

The 3-VPC shared/data/apps layout, peering, module scaling.

count vs for_each

The rename trap; stable keys win identity; 5 copy-ready patterns.

Outputs

5 types, sensitive vs ephemeral vs Vault, cross-stack pipelines.

Walkthrough

5 files, 3 workspaces, isolation proof, lab checklist.

ALB Walkthrough

Public + internal ALBs, quartet per tier, SG chaining.

★ Capstone

Full multi-env/multi-VPC build, end to end, ready to run.

Variables — Precedence, Types, Validation

Variables are how values get *in*. Seven mechanisms feed one declaration — later wins. Memorize the ladder, then the types, then validation. Everything downstream consumes this.

🔑 THE LADDER (LOWEST → HIGHEST)

TF_VAR_* env < **terraform.tfvars** < ***.auto.tfvars (lexical)** < **-var-file** < **-var**. default is the fallback when nothing provides a value.

Prove it once: set all four to different values, plan with **-var** — CLI wins.

§1 Passing mechanisms — when each earns its place

```
# CLI flags (highest) – envs + one-off tests
terraform apply -var="env=prod" -var="count=3"
terraform apply -var-file=env/prod.tfvars
terraform plan -var="env=dev" -out=tfplan && terraform apply tfplan

# Env vars (CI's favorite) – no files, no mounts
export TF_VAR_env="prod" TF_VAR_count=5
# complex types need HCL syntax: TF_VAR_tags='{ "Env": "prod" }'
# quoting mistakes here are the #1 CI variable bug

# terraform.tfvars (auto-loaded) – single-dev laptop only
env = "dev" count = 2 region = "us-east-1"
# multi-dev? rename per person + -var-file. NEVER commit secrets:
# commit terraform.tfvars.example, gitignore the real one.

# *.auto.tfvars (lexical overlay) – base + per-dev layers, no flags
# env/base.auto.tfvars → env/override.auto.tfvars (o wins per key)
# trap: a forgotten *.auto.tfvars silently beats your -var-file
```

§2 Type system — declare shape, catch errors at plan

```
variable "env" { type = string; default = "dev" }
variable "count" { type = number; default = 1 }
variable "enabled" { type = bool; default = true }
variable "tags" { type = map(string); default = {} } # NEVER null for maps
variable "azs" { type = list(string); default = ["us-east-1a"] }
variable "items" { type = set(string); default = [] } # for_each, never count
variable "server" { type = object({ name=string, size=number }) }
variable "pair" { type = tuple([string, number]) }
```

Three underused attributes: **sensitive** hides CLI output (state still plaintext); **nullable = false** makes explicit null fall back to default (saves `merge()`); **ephemeral** never persists to state — one-shot secrets only.

§3 Validation — fail at plan, not at 2 AM

```
variable "env" {
  type = string
  validation { condition = contains(["dev","staging","prod"], var.env)
               error_message = "must be dev/staging/prod" }
}
variable "cidr" {
  type = string
  validation { condition = can(cidrhost(var.cidr, 0))
               error_message = "must be a valid CIDR" }
}
```

Multiple validation blocks allowed — all must pass. Split complex rules so the error names the exact problem. Prod rule: required vars (no default) + strict validation; secrets from Vault/SSM, never files.

Functions, Dynamic Blocks, Secrets

Declarations get values in; **functions shape them**, **dynamic blocks repeat them**, and **data sources keep secrets out of variables entirely**. Test anything here instantly: `echo 'format("vpc-%s", var.env)' | terraform console`.

§1 The 20 functions you'll actually call

String

```
format("node-%02d-%s",3,var.env)
join(", ", var.azs)
split(", ", "a,b,c")
upper(var.env) / lower(var.env)
substr(var.project, 0, 8)
replace(var.name, "_", "-")
trimspace(var.input)
```

Collection

```
merge(var.tags,{Env=var.env})
concat(var.a, var.b)
distinct(concat(var.a,var.b))
flatten(["a","b"],["c"])
keys(var.amis)/values(var.amis)
lookup(var.amis,var.region,"fb")
contains(var.azs,"us-east-1a")
length(var.azs)
```

CIDR & safe

```
cidrsubnet("10.0.0.0/16",8,k)
cidrhost("10.0.1.0/24",5)
max(var.min,2)/min(var.max,10)
try(var.opt.attr,"default")
can(regex("^t[23]\\. ",var.type))
coalesce(var.a,var.b,"fb")
one(aws_x.maybe[*].id)
```

Prefer `length()` / `contains()` / `lookup()` with a default over `try()` — cheaper to read. `merge()` is shallow (top-level keys only). `try()` never hides errors outside validation — don't use it to paper over real bugs.

§2 Dynamic blocks — N rules from one variable

```
variable "ingress_rules" {
  type = list(object({ desc=string from=number to=number
                      proto=string cidrs=list(string) }))

  default = [
    { desc="http", from=80, to=80, proto="tcp", cidrs=["0.0.0.0/0"] },
    { desc="https", from=443, to=443, proto="tcp", cidrs=["0.0.0.0/0"] } ]
}

resource "aws_security_group" "web" {
  vpc_id = aws_vpc.main.id
  dynamic "ingress" {
    for_each = var.ingress_rules
    content {
      description = ingress.value.desc
      from_port = ingress.value.from   to_port = ingress.value.to
      protocol = ingress.value.proto   cidr_blocks = ingress.value.cidrs
    }
  }
}
```

Add a rule = append one object to `tfvars`, zero HCL edits. Same shape drives ALB listener actions and IAM policy statements.

§3 Secrets — variables carry the path, never the value

```
# SSM (AWS-native, no extra infra)
data "aws_ssm_parameter" "db_pw" {
  name = "${var.env}/db/password" with_decryption = true }
variable "db_pw_path" { default = "/dev/db/password" } # path only!

# Secrets Manager (rotation-friendly)
data "aws_secretsmanager_secret_version" "db" { secret_id = "prod/db" }
locals { db = jsondecode(data.aws_secretsmanager_secret_version.db.secret_string) }

# Vault (multi-cloud) — same idea via data "vault_kv_secret_v2"
```



State holds plaintext regardless — bucket SSE + tight IAM is non-negotiable. CI: pass the secret's **path/ARN** as the variable; let Terraform read the value at apply time.



Networking — Protocols, CIDR, Routing

Every AWS resource that takes a CIDR uses one format. Learn TCP/UDP, the handshake failures, CIDR math, and routing once — then every `aws_security_group` and `aws_route_table` is syntax.

🔑 DEBUG ORDER

Route table → SG → NACL. "Can't reach the internet" is the route table 80% of the time. Memorize this order before \$4.

§1 TCP vs UDP + the handshake that explains every error

	TCP	UDP
Style	Connection-oriented, reliable, ordered	Fire-and-forget, fast, lossy
Uses	HTTP/SSH/DB/file transfer	DNS, video, gaming, VoIP
Broken	"refused" / timeout / RST	Silent loss
Terraform	<code>protocol="tcp"</code>	<code>protocol="udp"</code>

SYN → | ← SYN-ACK | ACK → | DATA flows
 RST = reached, nothing listening (wrong port / app down)
 TIMEOUT = packet dropped (SG deny, NACL deny, route missing)

Top bug: opening TCP/80 when the server listens on UDP (custom metrics endpoint). Mismatched protocol = silent "no connection".

§2 CIDR math — the only operation that matters

```

/32 = 1 IP · /24 = 256 (251 usable) · /16 = 65,536 · /8 = 16M
/16 + 8 newbits = /24 → 2^8 = 256 subnets
/16 + 4 newbits = /20 → 2^4 = 16 subnets
locals {
  pub  = [for k in range(3) : cidrsubnet("10.0.0.0/16", 8, k)]      # .0 .1 .2
  priv = [for k in range(3) : cidrsubnet("10.0.0.0/16", 8, k + 10)] # .10 .11 .12
  db   = [for k in range(3) : cidrsubnet("10.0.0.0/16", 8, k + 20)] # gaps = growth
}

```

Reserved per subnet (5): .0 network, .1 router, .2 DNS, .3 future, last = broadcast. Pick `10.x.0.0/16` per env so peered VPCs never overlap.

§3 Routing — what actually delivers the packet

```

# public: everything else → IGW
resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }
}
# private: everything else → NAT (outbound only)
resource "aws_route" "priv_nat" {
  route_table_id = aws_route_table.private.id
  destination_cidr_block = "0.0.0.0/0"
  nat_gateway_id = aws_nat_gateway.main.id
}

```

Longest-prefix match wins. Every subnet uses exactly one route table (explicit or main). SG allows port 80 but no `0.0.0.0/0 → igw?` Packets go nowhere. Tiers are code, not magic: public = igw route + `map_public_ip_on_launch`; private = nat route; database = local-only, no default route.

SG vs NACL, DNS, Endpoints, Peering

The #1 interview question lives here: **stateful vs stateless**. SGs auto-allow return traffic; NACLs need both directions explicit. Then DNS flags, endpoints that skip NAT, and peering's silent DNS bug.

§1 SG vs NACL — the table to memorize

	Security Group	Network ACL
Level	Instance / ENI	Subnet
Stateful?	Yes — return auto-allowed	No — both directions explicit
Rules	Allow only	Allow + deny (numbered 1–32766)
Default	Allow out, deny in	Allow all both ways
Use	Default for everything	Block a bad IP range; rare

```
# NACL: HTTPS in + EPHEMERAL return (1024-65535) – SG never needs this
resource "aws_network_acl_rule" "https_in" {
  network_acl_id = aws_network_acl.main.id rule_number = 100
  egress = false protocol = "tcp" rule_action = "allow"
  cidr_block = "0.0.0.0/0" from_port = 443 to_port = 443
}
resource "aws_network_acl_rule" "ephemeral_out" {
  network_acl_id = aws_network_acl.main.id rule_number = 110
  egress = true protocol = "tcp" rule_action = "allow"
  cidr_block = "0.0.0.0/0" from_port = 1024 to_port = 65535
}
```

§2 DNS flags + endpoints that skip NAT

```
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
  enable_dns_support = true # .2 resolver in every subnet
  enable_dns_hostnames = true # public IPs get public DNS names
}
# Gateway endpoint for S3 – FREE, keeps traffic on AWS backbone
resource "aws_vpc_endpoint" "s3" {
  vpc_id = aws_vpc.main.id
  service_name = "com.amazonaws.us-east-1.s3"
  vpc_endpoint_type = "Gateway"
  route_table_ids = [aws_route_table.private.id]
}
```

DNS off = no discovery, no Route 53 private zones, no nslookup between instances. Endpoints: add lines, save NAT charges, gain latency. Format: `com.amazonaws.<region>.<service>`.

§3 Peering + the silent DNS bug

```
resource "aws_vpc_peering_connection" "a_to_b" {
  vpc_id = aws_vpc.a.id peer_vpc_id = aws_vpc.b.id }
# routes in BOTH tables – peering never auto-propagates
resource "aws_route" "a_to_b" {
  route_table_id = aws_route_table.a.private.id
  destination_cidr_block = aws_vpc.b.cidr_block
  vpc_peering_connection_id = aws_vpc_peering_connection.a_to_b.id }
# THE SILENT BUG – DNS does NOT cross peering by default:
resource "aws_vpc_peering_connection_options" "a" {
  vpc_peering_connection_id = aws_vpc_peering_connection.a_to_b.id
  requester { allow_remote_vpc_dns_resolution = true } }
# + mirror block with acceptor { ... } on side B
```

⚠ Without the flag on **both** sides: app in A can't resolve private hostnames in B — timeout while SG/route checks pass. Peering is 1:1, no transitive routing; past 3–4 VPCs use Transit Gateway.

§4 Interview answers

- **Public vs private — actual difference?** Route table (igw vs nat) + `map_public_ip_on_launch`. Nothing else.
- **Endpoint vs NAT?** NAT = generic egress (paid). Endpoint = one AWS service, cheaper, faster. S3/DynamoDB always via endpoint.
- **Reserved IPs?** 5 per subnet (.0 .1 .2 .3 broadcast). /24 = 251 usable.

Workspaces — CLI, Patterns, Proof

Workspaces are **named sub-folders inside one state backend**. Same code, same backend, one state file per env. The default workspace is free and implicit — create named ones **before the first apply**.

§1 Command reference

```
terraform workspace list      # * marks active
terraform workspace new dev   # create + switch (errors if exists)
terraform workspace select prod # switch
terraform workspace show      # prints name only
terraform workspace delete dev # safe only when state empty
terraform workspace delete dev -force
# → label gone, STATE FILE INTACT on backend. rm the S3 object yourself.
# names: letters digits - _ . no spaces/slashes . default is reserved
```

§2 Three in-code patterns via terraform.workspace

```
# A — sizing per workspace (textbook)
locals {
  sizing = { default={count=1,type="t3.micro"} dev={count=1,type="t3.micro"}
             staging={count=2,type="t3.small"} prod={count=3,type="t3.small"} }
  current = local.sizing[terraform.workspace]
}
resource "aws_instance" "cluster" {
  count = local.current.count instance_type = local.current.type
  tags = { Env = terraform.workspace }
}

# B — common + per-env overlay (lookup default defeats KeyError)
locals {
  tags = merge({ManagedBy="terraform"},
    lookup({dev={Env="dev"} prod={Env="prod"}}, terraform.workspace,
      {Env=terraform.workspace}))
}

# C — toggle per workspace
resource "aws_cloudwatch_log_group" "verbose" {
  count = contains(["dev","staging"], terraform.workspace) ? 1 : 0
  retention_in_days = terraform.workspace == "dev" ? 1 : 14
}
```

§3 Isolation proof — run once, win every argument

```
terraform init
terraform workspace new dev      && terraform apply -auto-approve # 1x
terraform workspace new staging && terraform apply -auto-approve # 2x
terraform workspace new prod    && terraform apply -auto-approve # 3x
aws s3api list-objects-v2 --bucket tfstate --prefix env/ --query 'Contents[].Key'
# env/dev/terraform.tfstate · env/staging/terraform.tfstate · env/prod/...
terraform workspace select dev && terraform plan # → 0/0/0, own state only
```

Each workspace's graph knows only its resources. Cross-workspace reads need `terraform_remote_state` (legacy) or module outputs — workspaces alone share nothing.



Workspaces in Production — CI, Sharing, Gotchas

Workspaces are easy to learn, easy to misuse. This page: the decision table, the CI shape that removes human error, cross-workspace sharing, and the 3 gotchas that bite first.

§1 Workspaces vs folders vs -var-file

	Workspaces	Separate folders	Single -var-file
State isolation	Yes, per env	Yes, full blast-radius	No — DANGEROUS
Per-env backends	No	Yes (accounts/regions)	N/A
"Which env?"	Hidden (workspace show)	Visible (cwd)	Visible (in command)
Locking	One lock, all envs	One lock per state	One lock, all
Best for	Solo/small, one account	Large/regulated, per-account	Throwaway only

§2 CI — never let a human type select prod

```
# plan dev+staging in parallel on PR; prod only after merge to main
env: { TF_WORKSPACE: ${ matrix.env } } # pipeline sets it, not fingers
run: terraform plan -var-file=envs/${ matrix.env }.tfvars -out=tfplan
# apply-prod: environment: production (repo approval gate)
env: { TF_WORKSPACE: prod }
run: terraform apply -var-file=envs/prod.tfvars tfplan
```

§3 Sharing across workspaces

A — remote_state (legacy)

```
data "terraform_remote_state" "net" {
  backend = "s3"
  config = { bucket="tfstate"
    key="env/prod/network/terraform.tfstate"
    region="us-east-1" } }
```

Workspace baked into key; needs bucket IAM. Works, marked legacy.

B — module output (modern)

```
module "network" {
  source="terraform-aws-modules/vpc/aws"
  version="5.1.0" name="prod"
  cidr="10.0.0.0/16" }
```

Each workspace provisions its own copy. No cross-state reads.

§4 The 3 gotchas

G1. default workspace. First init+apply lands there, unprefixed key. Fix: **workspace new before first apply.**

G2. Shared lock. One table locks all states in the bucket — no parallel applies. Fix: **separate buckets per env.**

G3. Delete keeps state. Label gone, S3 object stays. Fix: **aws s3 rm the key yourself.**



Multi-Env — Workspaces vs Folders vs Buckets

Three layouts for N envs, one VPC each. Pick by blast radius: solo → workspaces, team → folders, regulated → separate buckets. CIDRs must never overlap — enforce with a matrix in code.

§1 Layout B1 — workspaces (1 folder, 1 backend, many states)

```

locals {
  cidrs = { dev="10.0.0.0/16" staging="10.10.0.0/16" prod="10.20.0.0/16" }
}
resource "aws_vpc" "main" {
  cidr_block = local.cidrs[terraform.workspace]
  tags = { Env = terraform.workspace }
}
# terraform workspace new dev && terraform apply (x3 envs, 3 states, 3 CIDRs)

```

Best for solo, single account. Same-bucket blast radius is the price of simplicity.

§2 Layout B2 — separate folders (1 folder per env)

```

envs/dev/main.tf           # no env logic: cidr 10.0.0.0/16, Env=dev
envs/dev/backend-dev.hcl   # bucket=tfstate key=env/dev/terraform.tfstate
envs/staging/... envs/prod/...
cd envs/dev && terraform init -backend-config=backend-dev.hcl && terraform apply

```

Best for teams: per-env lock, per-env IAM, CI matrix over dirs. A local module kills the duplication (see ch-04b §4).

§3 Layout B3 — separate buckets (production grade)

```

# envs/dev/backend-dev.hcl → bucket = "tfstate-dev"
# envs/prod/backend-prod.hcl → bucket = "tfstate-prod" (different account)
# Dev apply CANNOT touch the prod bucket (different IAM). Survives audits.

```

§4 Decision flowchart

```

1 account · 1 env · study ..... Pattern A (single folder)
1 account · N envs ..... B1 workspaces / B2 folders
1 account · N envs · audit ..... B3 separate buckets
N envs · 2+ VPCs each ..... Pattern C (next page)
N accounts · N envs ..... C3 one account per env
5+ envs · shared config ..... Terragrunt

```

Multi-VPC — The 3-VPC Layout

Real orgs run **shared + data + apps** VPCs per env: ALB/NAT live in shared, RDS/cache in data (no internet), compute in apps. Peered together, isolated at the SG level.

§1 CIDR matrix — overlap is a plan-time error, not a runtime surprise

```
locals {
  cidr_matrix = {
    dev    = { shared="10.0.0.0/16" data="10.1.0.0/16" apps="10.2.0.0/16" }
    staging = { shared="10.10.0.0/16" data="10.11.0.0/16" apps="10.12.0.0/16" }
    prod   = { shared="10.20.0.0/16" data="10.21.0.0/16" apps="10.22.0.0/16" }
  }
  my = local.cidr_matrix[terraform.workspace][var.vpc_role] # role: shared/data/apps
}
resource "aws_vpc" "this" {
  cidr_block = local.my_cidr
  tags = { Env = terraform.workspace Role = var.vpc_role }
}
```

§2 Peering + routes — explicit on both sides, no transitive

```
resource "aws_vpc_peering_connection" "shared_apps" {
  vpc_id = aws_vpc.shared.id peer_vpc_id = aws_vpc.apps.id
  auto_accept = true # same account; cross-account needs accept_request
  tags = { Pair = "shared-apps" }
}
resource "aws_route" "apps_to_shared" {
  route_table_id = aws_vpc.apps.main_route_table_id
  destination_cidr_block = aws_vpc.shared.cidr_block
  vpc_peering_connection_id = aws_vpc_peering_connection.shared_apps.id
}
# SGs chain by REFERENCE inside a VPC, by CIDR across VPCs:
# apps_ec2 ingress: source = shared_alb.id (same VPC — never 0.0.0.0/0)
```

§3 Scale with a module — 9 lines for the full pattern

```
# envs/prod/main.tf — one module block per role, different CIDRs
module "shared" { source="../../modules/vpc-with-peering"
  cidr="10.0.0.0/16" role="shared" az_count=3 }
module "data" { source="../../modules/vpc-with-peering"
  cidr="10.10.0.0/16" role="data" az_count=3 }
module "apps" { source="../../modules/vpc-with-peering"
  cidr="10.20.0.0/16" role="apps" az_count=3 }
```

Change the module → every env re-applies (per-state tracking, no cross-env pollution). State layout: s3://tfstate-prod/shared|data|apps/terraform.tfstate — one bad eks/ change can't touch data/.

§4 Gotchas interviews test

- **Transitive routing doesn't exist.** apps→shared→data needs a direct apps→data peering, or TGW.
- **DNS over peering:** `allow_remote_vpc_dns_resolution` on both sides (ch-02b §3).
- **SG refs die across VPCs** — use CIDR sources cross-VPC, SG refs inside.
- **State in default workspace** — workspace new before first apply, always.



count vs for_each — The Rename Trap

Both copy a resource N times. **count = index identity**, **for_each = key identity**. Remove from the middle and count shifts every later copy; for_each kills only the removed key. That's the whole chapter in two sentences — the rest is proof.

THE RULE

for_each for anything with a meaningful identity (servers by name, subnets by AZ). **count** for "N identical copies" or 0/1 toggles. Never mix both in one block.

§1 The difference, demonstrated

count — index-based

```
resource "aws_instance" "web" {
  count = 3
  tags = { Name = "web-${count.index}" }
}
# → web[0] web[1] web[2]
# count 3→2 destroys web[2].
# Remove the MIDDLE? Later indexes
# SHIFT — downstream refs break.
```

for_each — key-based

```
resource "aws_instance" "web" {
  for_each = toset(["web-a", "web-b"])
  tags = { Name = each.key }
}
# → web["web-a"] web["web-b"]
# Remove "web-c": ONLY it dies.
# Rename: old key dies, new key
# born, everything else untouched.
```

§2 5 copy-ready patterns

```
# 1 — subnets per AZ (map: AZ → netnum, best of both)
resource "aws_subnet" "public" {
  for_each = { for i, az in var.azs : az => i }
  cidr_block = cidrsubnet("10.0.0.0/16", 8, each.value)
  availability_zone = each.key
}
# 2 — IAM users from a list (set: stable identity)
resource "aws_iam_user" "team" {
  for_each = toset(var.users) # add "noor" → only noor created
  name = each.key
}
# 3 — N identical workers (count earns it here)
resource "aws_instance" "worker" {
  count = 5
  tags = { Name = "worker-${count.index}" } # 4 destroys worker[4]
}
# 4 — conditional 0/1 + safe read
resource "aws_cloudwatch_log_group" "v" {
  count = var.enable_monitoring ? 1 : 0
}
output "log_group" { value = one(aws_cloudwatch_log_group.v[*].arn) }
# 5 — for_each needs set/map, never a bare list:
# for_each = toset(var.list) or { for x in var.list : x => x }
```

§3 Indexing without the Invalid-index error

```
output "ip_or_null" { value = try(aws_instance.web[0].public_ip, null) }
output "by_name"    { value = lookup(aws_instance.team, "ahmed", null) }
```

for_each on unknown-at-plan values fails — use count there, or refactor. one() returns the single element or null — perfect for 0/1 resources.



Outputs — The Production Contract

An output is a **named export from state** — to modules, other stacks, CI, humans. `outputs.tf` is the public interface reviewers read first. Five types, one security model, three consumption patterns.

§1 Anatomy + the 5 types

```

output "vpc_id" { value = aws_vpc.main.id } # 1 · scalar
output "public_ips" { value = aws_instance.web[*].public_ip } # 2 · list
output "by_name" { # 3 · map (friendliest for curl + downstream)
  value = { for k, inst in aws_instance.web : k => inst.public_ip } }
output "db_password" { # 4 · sensitive (CLI-masked, state PLAINTEXT)
  value = aws_db_instance.main.password sensitive = true }
output "deploy_token" { # 5 · ephemeral (never in state; same-apply read only)
  value = aws_secretsmanager_secret_version.cur.secret_string
  ephemeral = true }
# + description (always set it) · depends_on (rare) · precondition (underused §3)

```

§2 Security model — the interview answer

	CLI hidden?	In state?	Real secrecy?
Plain	No	Yes	No
sensitive	Yes (masked)	Yes, plaintext	No — masking only
ephemeral	Yes	No	One-shot only
Vault/SSM/SM	Via provider	Encrypted at rest	Yes — the true story

§3 Consumption — module, stack, CI

```

# module → module (standard): module.vpc.private_subnets[0]
# stack → stack (legacy remote_state):
data "terraform_remote_state" "network" {
  backend = "s3"
  config = { bucket="tfstate-prod" key="network/terraform.tfstate" } }
# → data.terraform_remote_state.network.outputs.private_subnet_ids[0]
# source stack MUST run first — CI job graph enforces it.
# CI: terraform output -json | jq -r .vpc_id.value (mask secrets!)
# plan archaeology: terraform show -json tfplan | jq '.output_changes'

```

§4 Guardrails — precondition + naming

```

output "cluster_size" {
  value = length(aws_instance.cluster)
  precondition { condition = length(aws_instance.cluster) >= 2
    error_message = "Need ≥2 instances for HA" }
}

```

Name outputs `web_instance_ips`, not `ips` — consumers shouldn't guess. Never echo a sensitive output; pipe to a secret store.



End-to-End Project Walkthrough

Scaffold → code → 3 workspaces → isolation proof. A tiny real cluster: 1 VPC/env, public+private subnets, N instances serving Hello from <env>-<index>. 30 min on the emulator.

§1 Layout — 5 files, resources apart from providers

```

study-cluster/
├── versions.tf      # pins: TF ≥1.6, aws ~> 5.0 (+ lock file, COMMIT it)
├── variables.tf     # env_override · sizing map · cidrs map (non-overlapping!)
├── providers.tf     # provider blocks, one per cloud
├── main.tf          # VPC → subnets → SG → AMI → instances
└── outputs.tf       # vpc_id · subnets · ips-by-name · count · env

```

§2 The core trick — env from workspace, guarded

```

locals {
  env_name = var.env_override != "" ? var.env_override : terraform.workspace
  effective_env = local.env_name == "default" ? "dev" : local.env_name
  effective     = var.sizing[local.effective_env] # {count, type}
  vpc_cidr      = var.cidrs[local.effective_env]  # 10.0 / 10.10 / 10.20
  azs = { for i, az in slice(data.aws_availability_zones.av.names,0,3) : az => i }
  node_names = [for i in range(local.effective.count) :
    format("%s-node-%02d", local.effective_env, i)]
}

resource "aws_subnet" "public" {
  for_each = local.azs # stable identity per AZ
  cidr_block = cidrsubnet(aws_vpc.this.cidr_block, 8, each.value)
  availability_zone = each.key map_public_ip_on_launch = true
}

resource "aws_instance" "node" {
  for_each = toset(local.node_names)
  subnet_id = values(aws_subnet.public)[0] # simplified; prod balances 3
  user_data = <<-EOT
    #!/bin/bash
    apt-get install -y nginx
    echo "Hello from ${each.key}" > /var/www/html/index.html
  EOT
}

```

§3 Run it — init once, workspace per env, prove isolation

```

terraform init # providers + .terraform.lock.hcl
terraform workspace new dev staging prod # one command each
terraform workspace select dev && terraform apply # 1x → curl: Hello from dev-node-00
terraform workspace select staging && terraform apply -auto-approve # 2x
terraform workspace select prod && terraform apply -auto-approve # 3x
# PROOF: each env plans clean against its own state:
terraform workspace select dev && terraform plan # → 0/0/0
# destroy one env, others untouched: select staging && terraform destroy

```

Remote backend for teams: backend "s3" with key = "study-cluster/terraform.tfstate" — workspaces prefix env/<name>/ automatically.

terraform output -json | jq .instance_count.value per env must read 1 / 2 / 3.



ALB Walkthrough — Two Tiers, One Codebase

A public ALB in front of two frontend instances, an internal ALB in front of two backend instances — 18 resources, one `main.tf`, three `envs` via workspaces. Every ALB stack is the same **quartet**: load balancer → target group → listener → attachment.

§1 The canonical quartet (public tier)

```

resource "aws_lb" "public" {
  name = "${local.name_prefix}-pub-alb"
  load_balancer_type = "application"
  internal = false # ← internet-facing; true = internal tier
  security_groups = [aws_security_group.public_alb.id]
  subnets = aws_subnet.public[*].id
}

resource "aws_lb_target_group" "public" {
  name = "${local.name_prefix}-pub-tg" port = 80
  protocol = "HTTP" target_type = "instance" vpc_id = aws_vpc.this.id
  health_check { path = "/" port = "traffic-port" matcher = "200"
    interval = 30 timeout = 5 healthy_threshold = 2 unhealthy_threshold = 3 }
}

resource "aws_lb_listener" "public_http" {
  load_balancer_arn = aws_lb.public.arn port = 80 protocol = "HTTP"
  default_action { type = "forward"
    target_group_arn = aws_lb_target_group.public.arn }
}

resource "aws_lb_target_group_attachment" "public" {
  count = length(aws_instance.frontend) # 2 + 2, same count as the fleet
  target_group_arn = aws_lb_target_group.public.arn
  target_id = aws_instance.frontend[count.index].id port = 80
}

```

Internal tier = same quartet with `internal = true`, swapped SG/subnets/targets. Health-check rule of thumb: `interval ≥ 2 × timeout`, thresholds summing to 5 for fast convergence.

§2 SG chaining — ALB → EC2 by reference, never CIDR

```

# public_alb: TCP 80/443 from 0.0.0.0/0 (internet-facing)
# frontend_ec2: TCP 80 from public_alb SG ONLY (not 0.0.0.0/0)
# internal_alb: TCP 80 from frontend_ec2 SG + VPC CIDR (defense in depth)
# backend_ec2: TCP 80 from internal_alb SG ONLY (internal traffic only)
ingress { from_port = 80 to_port = 80 protocol = "tcp"
  security_groups = [aws_security_group.public_alb.id] }

```

SGs referencing each other form a chain Terraform topologically orders — no `depends_on` needed.

§3 The elbv2 gotcha + workspace naming

⚠️ `aws_lb` needs the **elbv2** endpoint — classic `elb` serves CLBs only. Missing `elbv2` line = ALB calls silently hit real AWS with emulator creds set. Always redirect `ec2`, `elb`, `elbv2`, `iam`, `sts`, `s3` together via dynamic "endpoints" on a null-or-URL variable.

```

locals {
  name_prefix = "${var.project}-${terraform.workspace}" # onext-dev-...
  common_tags = { Project = var.project
    Environment = terraform.workspace ManagedBy = "terraform" }
}
# run-alb.sh dev → 22 resources in dev state; staging/prod identical, isolated

```

Proof it's correct, not one-shot: re-run `plan` → **No changes**. Outputs (`public_alb_dns_name` + `_zone_id` for Route53 alias, TG ARNs, instance IDs) feed downstream stacks.

★ Lab 1 — Multi-Env & Multi-VPC Build

Stitches every chapter: **variables** → **workspaces** → **3-VPC layout** → **for_each** → **outputs** → **destroy**. Two tabs — Tab 2 builds, Tab 1 observes. Emulator-ready; same code hits real AWS by deleting endpoints + `skip_*` lines.

▣ Build chain (8 resources, one file)

```
terraform {
  required_providers { aws = { source = "hashicorp/aws", version = "~> 5.0" } }
}
provider "aws" {
  region = "us-east-1"
  access_key = "test" secret_key = "test" # emulator dummy creds
  skip_credentials_validation = true
  skip_requesting_account_id = true
  endpoints { ec2 = "http://localhost:4566" } # delete for AWS
}
locals {
  cidrs = { shared="10.0.0.0/16" data="10.10.0.0/16" apps="10.20.0.0/16" }
}
resource "aws_vpc" "shared" { cidr_block = local.cidrs.shared
  tags = { Env=terraform.workspace Role="shared" } }
resource "aws_vpc" "data" { cidr_block = local.cidrs.data
  tags = { Env=terraform.workspace Role="data" } }
resource "aws_subnet" "pub" {
  for_each = { for i, az in slice(data.aws_availability_zones.av.names,0,2) : az => i }
  vpc_id = aws_vpc.shared.id
  cidr_block = cidrs[aws_subnet.pub].cidr_block
}
resource "aws_security_group" "web" {
  vpc_id = aws_vpc.shared.id
  ingress { from_port=443 to_port=443 protocol="tcp" cidr_blocks=["0.0.0.0/0"] }
  egress { from_port=0 to_port=0 protocol="-1" cidr_blocks=["0.0.0.0/0"] }
}
resource "aws_instance" "web" {
  for_each = toset(["a1","a2","b1","b2"])
  ami = "ami-0c55b159cbf4e1f0" instance_type = "t3.micro"
  subnet_id = values(aws_subnet.pub)[0]
}
output "ips" {
  value = { for k, i in aws_instance.web : k => i.private_ip } }
```

▣ Two-terminal execution

👁️ TAB 1 • OBSERVE

```
floci start
eval "$(floci env)"
watch aws ec2 describe-vpcs \
  --query 'Vpcs[].CidrBlock'
```

🇺🇸 TAB 2 • BUILD

```
terraform init
terraform workspace new dev
terraform apply -auto-approve
terraform output -json | jq .ips
terraform destroy -auto-approve
```

🎓 WHAT THIS PROVED

Provider auth → per-workspace CIDRs → `for_each` identity → SG → outputs → isolated destroy. Repeat for staging/prod with the matrix from ch-04b.



Cheat · Quiz · Glossary · Resources

The story in one page: commands, quiz, glossary, resources. Tear this page out (mentally) and keep it by your terminal.

▣ Cheat sheet — the commands

Command	What it does
<code>terraform init</code>	Install providers, init backend, lock versions
<code>terraform plan -out=tf</code>	Preview + save (reviewed == executed)
<code>terraform apply tf</code>	Execute exactly the saved plan
<code>terraform output -json</code>	All exports, CI-shaped (<code> jq .x.value</code>)
<code>workspace new/select/show</code>	Create / switch / print active env
<code>terraform console</code>	REPL — test any function/var instantly
<code>terraform destroy</code>	Tear down THIS workspace only

▣ Quiz — 6 questions

1. Order the ladder: `-var`, `auto.tfvars`, `tfvars`, `TF_VAR_`, `default`.

A: `-var` > `-var-file` > `auto.tfvars` > `tfvars` > `TF_VAR_` > `default`.

2. Why does removing a middle count item shift identity?

A: Index-based — later items renumber. `for_each` keys don't.

3. SG allows 443 but curl times out. First check?

A: Route table (0.0.0.0/0 → `igw/nat`), then NACL ephemeral ports.

4. Does `sensitive=true` encrypt state?

A: No — CLI masking only. State stays plaintext.

5. Peering up but cross-VPC DNS fails?

A: `allow_remote_vpc_dns_resolution` on BOTH sides.

6. Deleted workspace — is the state gone?

A: No — label only. `rm` the S3 key yourself.

▣ Glossary — 12 terms

workspace — named state scope under one backend.

precedence ladder — CLI > `auto.tfvars` > `tfvars` > `env` > `default`.

validation — plan-time input guard with `error_message`.

dynamic block — N nested blocks from one variable.

ephemeral — never persisted; one-shot secrets.

longest-prefix match — most specific route wins.

stateful / stateless — SG auto-returns; NACL needs both ways.

endpoint — private path to one AWS service.

rename trap — count re-indexes; `for_each` keys hold.

one() — single-or-null reader for 0/1 resources.

precondition — output sanity gate at eval time.

TF_WORKSPACE — CI's non-interactive env switch.

▣ Resources — keep these handy

Course deck (this book is built on it): Eng. Mohamed Magdy — Terraform Level 1 + 2

Official: registry.terraform.io · developer.hashicorp.com/terraform · github.com/hashicorp/terraform

Learn: [learn-terraform](#) (free labs) · [discuss](#) (forum)

Local emulator: floci.io (free MIT, port 4566)

Exam: HashiCorp Certified — Terraform Associate